

DRAFT v1.0

PROTOTYPE SCOPE

Software Requirements Specification

SecureMailScope — AI-Assisted Cryptographic Security Posture
Assessment for Secure Email Communications (SIH PS 26159)

Document	SRS — Prototype / Hackathon build
Applies to	36–48 hour SIH prototype, single-node deployment
Std. basis	Structured per IEEE 830 sections, trimmed for hackathon scope

1. Introduction

1.1 Purpose

This SRS defines exactly what the SecureMailScope prototype must do to be considered complete for demonstration. It scopes the system to what is achievable and demonstrable within a hackathon timeframe, while satisfying every deliverable listed in PS 26159. Anything not listed here is explicitly out of prototype scope (see 2.3).

1.2 Scope

The prototype accepts a PCAP file containing SMTP/IMAP/POP3 traffic, passively analyzes it for cryptographic weaknesses, computes a per-session risk score using the SCoRE algorithm, flags anomalies, and produces JSON, PDF and HTML reports plus an interactive dashboard. It does not connect to any live server, does not decrypt payload content, and does not require internet access at runtime.

1.3 Definitions, abbreviations

Term	Meaning
PCAP	Packet capture file (input artifact)
Session	One reconstructed SMTP/IMAP/POP3 TCP conversation

Finding	One detected weakness within a session, from the rule engine
SCoRE	Session Cryptographic Overall Risk Evaluator — this project's scoring algorithm (§4.1)
SOC	Security Operations Center — a target user class

2. Overall Description

2.1 Product perspective

Standalone system, not a plugin to existing tools. Consumes PCAP files as its only input format for the prototype. Runs self-contained via Docker Compose. No dependency on any other NTRO or third-party system for the prototype phase.

2.2 User classes and characteristics

User class	Technical level	Primary need
SOC analyst	High (security background)	Fast triage — which session to look at first
Digital forensics investigator	High	Evidence-grade detail per session, exportable report
Enterprise mail admin	Medium	Plain-language remediation steps
Judge / evaluator (prototype-specific)	Variable	Clear demo of every PS deliverable in under 5 minutes

2.3 Design and implementation constraints (prototype)

- Single-node deployment only (no Kubernetes, no multi-region) — that is production scope, see PRD
- Max input tested: PCAPs up to ~2 GB
- No live network scanning of any kind — passive analysis of supplied files only
- No decryption of payload content under any circumstance
- TLS 1.3 sessions: certificate fields explicitly marked `"observable": false` where handshake is encrypted, rather than silently omitted
- Must run fully offline once built (no runtime internet dependency)

2.4 Assumptions and dependencies

- Input PCAPs are well-formed (not corrupted); malformed files are handled gracefully, not silently
 - Training data for ML models is the self-generated synthetic labelled corpus described in §5.2
 - Demo environment has Docker installed; no dependency on cloud accounts
-

3. System Features (Prototype)

Each feature below states what it must do, its inputs/outputs, and its done-criteria. This is the build checklist.

3.1 Capture Ingest & Stream Reassembly

Description: Accept a PCAP/PCAPNG file, validate it, reassemble TCP streams.

Functional requirements:

- FR-1.1: System shall accept files in `.pcap` and `.pcapng` formats
- FR-1.2: System shall reject and report malformed/truncated captures without crashing
- FR-1.3: System shall reconstruct bidirectional TCP streams handling out-of-order and retransmitted packets
- FR-1.4: System shall process files up to 2 GB within prototype hardware without exceeding 4 GB RAM (streaming reader)

Done when: a stream count and list of 4-tuples is produced for a sample 500 MB PCAP in under 60 seconds.

3.2 Protocol Identification & STARTTLS State Tracking

- FR-2.1: System shall classify each stream as SMTP, IMAP, POP3, or Other, using port + payload banner (not port alone)
- FR-2.2: System shall track STARTTLS negotiation state per session: offered / requested / completed / not-applicable
- FR-2.3: System shall flag `STRIPPING_SUSPECTED` when STARTTLS is offered but not completed and plaintext application data follows

Done when: a deliberately-stripped test session is correctly flagged, and a correctly-encrypted session is not false-flagged.

3.3 TLS Handshake & Certificate Analysis

- FR-3.1: System shall parse ClientHello (offered versions, ciphers, extensions, SNI) and ServerHello (negotiated version, cipher, key exchange) where visible
- FR-3.2: System shall extract X.509 certificate fields: subject, issuer, validity dates, public key algorithm and size, signature algorithm, SAN list
- FR-3.3: System shall validate certificate chain, expiry, self-signed status, hostname match, signature hash strength
- FR-3.4: For TLS 1.3 sessions, system shall report all still-visible metadata and explicitly mark encrypted fields as not-observable, never omit silently
- FR-3.5: System shall compute JA3/JA3S fingerprints per handshake

3.4 Rule Engine

- FR-4.1: System shall evaluate each session against a documented set of ≥ 30 deterministic weakness rules (deprecated TLS version, weak cipher, no forward secrecy, weak key size, expired/self-signed cert, etc.)
- FR-4.2: Every finding shall include: rule ID, severity, human-readable evidence, applicable compliance reference, and remediation text
- FR-4.3: Rule engine shall run and produce a complete finding set independent of whether the ML layer succeeds or fails

3.5 SCoRE Scoring & AI Layer

- FR-5.1: System shall compute a 0–100 score per session using the SCoRE algorithm (§4.1)
- FR-5.2: System shall classify each session into Critical/High/Medium/Secure per the SCoRE thresholds
- FR-5.3: System shall run an anomaly detector (Isolation Forest on JA3/JA3S + handshake features) and surface top-N anomalous sessions not caught by rules
- FR-5.4: System shall produce a per-score explanation trace (deterministic penalty, ML adjustment, anomaly penalty, urgency penalty) alongside SHAP top-3 factors for the ML component
- FR-5.5: System shall prioritize findings using $\text{priority} = \text{severity} \times \text{exposure} \times (1/\text{fix_effort})$, not severity alone

3.6 Reporting

- FR-6.1: System shall export results as JSON (schema-versioned)

- FR-6.2: System shall export a PDF report from the same template as the HTML report
- FR-6.3: System shall export a standalone HTML report
- FR-6.4: All three formats shall be derivable from a single underlying result object — no independent re-implementation per format

3.7 Dashboard

- FR-7.1: Dashboard shall show an overall posture score/grade for the analyzed capture
- FR-7.2: Dashboard shall show a sortable, filterable session table (by severity, protocol)
- FR-7.3: Dashboard shall show TLS version distribution, cipher heatmap, and certificate expiry timeline charts
- FR-7.4: Dashboard shall show a dedicated anomaly panel with ML reasoning
- FR-7.5: Dashboard shall support click-through drill-down from summary to per-session evidence

4. Algorithm Specification

4.1 SCoRE — Session Cryptographic Overall Risk Evaluator

Full formula, pseudocode and worked example are the authoritative spec for FR-5.1–FR-5.5. Implementers must match this exactly.

```
DP      = Σ w_i × min(1 + log2(scope_i + 1) / 4, 1.5)      # deterministic penalty
Blend   = DP × (1 + 0.4 × (p_ml - 0.5))                    # ML confidence fusion
AB      = anomaly_score × 25                                # anomaly boost
UB      = urgency_factor × 10                               # time-decay boost
Score   = clamp(100 - Blend - AB - UB, 0, 100)

Classes: 0-39 Critical | 40-69 High | 70-89 Medium | 90-100 Secure
```

Symbol	Source	Range
w_i	Rule engine (§3.4), expert-assigned per severity tier	5 / 10 / 20 / 35
scope_i	Count of sessions/hosts sharing the same root-cause finding	≥1
p_ml	Random Forest risk classifier output	0–1

anomaly_score	Isolation Forest / JA3 rarity score	0–1
urgency_factor	Time-decay, e.g. cert-expiry proximity	0–1

Non-negotiable implementation rule: the rule engine (DP) must be able to produce a complete, correct score with `p_ml` defaulted to 0.5 (neutral) and anomaly/urgency at 0 — i.e. the system must degrade gracefully to pure rule-based scoring if the ML service is unavailable. This is a testable requirement, not a suggestion.

5. Data Requirements

5.1 Core data entities

Entity	Key fields
Capture	id, filename, uploaded_at, packet_count, session_count
Session	id, capture_id, protocol, src/dst, starttls_state, tls_summary, score, risk_class
Finding	id, session_id, rule_id, severity, evidence, compliance_refs, remediation
Certificate	id, session_id, subject, issuer, validity, key_algo, key_bits, sig_algo, observable(bool)

5.2 Synthetic training dataset plan

~8 mail-server configuration profiles (plaintext → hardened TLS 1.3), ~50 sessions each, ~400 total labelled sessions. Ground truth derived directly from the deployed config, not manual labelling — this guarantees 100% label accuracy for the training set. Full generation steps: Docker Postfix/Dovecot profiles, tcpdump capture, swaks/openssl traffic generation, deliberately weak self-signed certs.

6. External Interface Requirements

Interface	Requirement
Upload API	<code>POST /captures</code> — multipart PCAP upload, returns <code>capture_id</code> + job status

Results API	<code>GET /captures/{id}/sessions</code> — paginated, filterable session list
Report API	<code>GET /captures/{id}/report.{json pdf html}</code>
Dashboard	Web SPA, served over HTTPS on localhost for prototype demo

7. Non-Functional Requirements (Prototype)

Category	Requirement
Performance	500 MB PCAP fully analyzed (ingest → score → report) in under 3 minutes on 8-core laptop
Reliability	Rule engine output must not depend on ML layer succeeding (\$4.1 box)
Portability	Entire prototype runs via single <code>docker compose up</code> , no manual setup steps
Security	No payload content is stored or logged; only protocol/crypto metadata persists
Usability	Dashboard usable by someone with no prior tool exposure within 2 minutes, unguided
Explainability	Every score must be traceable to a human-readable reason — no un-explained numbers anywhere in the UI

8. Verification

8.1 Primary use cases

1. **UC-1 Analyze a capture:** user uploads PCAP → system reassembles, parses, scores, reports → user views dashboard.
2. **UC-2 Investigate a critical session:** user clicks a Critical session → sees findings, SCoRE trace, remediation, evidence frame references.
3. **UC-3 Export for audit:** user downloads JSON + PDF + HTML for the same capture and confirms all three carry identical findings.
4. **UC-4 Detect an unknown anomaly:** user views the anomaly panel and finds a session flagged purely on fingerprint rarity, with no rule triggered.

8.2 Acceptance criteria (prototype "done" definition)

#	Criterion
AC-1	All 21 PS-26159 deliverables are demonstrable end-to-end
AC-2	STARTTLS stripping test case correctly flagged; a clean-TLS test case is not false-flagged
AC-3	SCoRE score + explanation trace visible for every session in the dashboard
AC-4	At least one anomaly is surfaced that no deterministic rule caught
AC-5	JSON, PDF and HTML exports all succeed and agree on findings for the same capture
AC-6	System runs with zero internet access at runtime
AC-7	Rule engine functions correctly with ML service forcibly disabled (graceful degradation proven)